

Owen Lopez

Professor Faulconer

ENC 3250

13 September 2026

Information Inventory and Content Strategy

Part One: Information Inventory & Content Strategy

My professional document will be about the code editor *Vim*, in particular the *VS Code* Extension for *Vim*. It will primarily focus on conveying basic usage of the code editor and the philosophy that supports its particular manner of editing.

Information Inventory

Must Know

1. *Core Navigation*: How to move the cursor.
2. *Modal Editing*: The fundamental philosophy behind the *Vim* code editor.
3. *Essential Modes*: How one operates the program.
4. *File Operations*: Saving and editing files, behaviors that are required to work with a project.

Useful to Know

1. *Advanced Navigation*: Specific mechanisms for moving efficiently throughout a file.
2. *Editing Operators*: Controls for editing entire blocks of text.
3. *Operator Combination*: High-level logic that allows for chains of commands.
4. *Search Operator*: Mechanisms for retrieving and editing specific parts of a document.

Don't Need to Know

1. *Terminal Vim*: Downloading and operating *Vim* from a terminal. The document will only explain *Vim* concepts that can be utilized anywhere.

2. *REGEX*: REGEX is an advanced pattern-matching tool that can be used in search strings; it is outside the scope of this document.
3. *Buffer Management*: The ability to open new visual panes, which display text concurrently.
4. *Configuration*: *Vim* contains an entire scripting and configuration language (in fact, two entire languages). These topics require a deeper analysis than can be provided in this document.

Content Strategy

The essential message of my document will be explaining how to properly utilize *Vim* in an introductory manner. As the program is very complex, it is essential that the guide covers only the most necessary features and concepts to allow a new user to start working within it. For this reason, the guide will focus on how to use an extension for the popular *VS Code* that utilizes only the core features of *Vim*, rather than the native terminal application.

The audience will require, in the following order, three key concepts in order to gain a basic understanding of *Vim*. They will begin with an overview of the Modal Editing Paradigm, which is the concept of editing data through utilizing specific tools, or modes. Applications like image editors or 3D modeling tools function in this manner, where users rotate between different tools that all behave differently. Once modal editing has been established, navigation throughout a project will be the first topic covered. This requires the explanation and memorization of the functions of a myriad of keys. Each key in *Vim* performs some action, and there are around ten keys to learn for basic navigation. Now that the audience can navigate, we will turn to actually performing some productive edits with basic operators. As with navigation, every key performs something different and thus has to be learned. Finally, the guide will end with file-level commands, like saving or exiting. These operate on a higher plane than a user's edits, with there being a core few that should be learned for basic usage.

There is an enormous amount of information that could be covered in the document, but I have decided to ignore the majority of it. Specifically, I will not cover configuration and the command line. *Vim* can be configured through a file written with a domain-specific language (Vimscript); learning an entire language in an introductory guide is impractical. The program is also typically interacted with in the terminal and the command line (CMD). As the audience will instead be learning through the *VS Code* extension, I will not mention any of the numerous command-line quirks that can be found when using native *Vim*.

I intend for the audience to download the extension and follow along with the guide themselves. Following along will provide the most efficient manner to learn the information presented. I have decided to use the *VS Code* extension due to my audience's lack of command-line knowledge. It is simpler to introduce *Vim* itself to begin with and then allow them to dive into the CMD when they are ready. Additionally, I will be framing *Vim* primarily as a code editing tool rather than a general-purpose text editor (I'm using it currently to write this document). This framing will appeal to the audience due to their programming background, as a means for enhancing their efficiency.

Part Two: The “So What?” Test

Table 1

The “So What?” Test: Audience Relevance of Core Vim Concepts

Information / Concept	Why Does My Audience Need to Know This? ("So What?")
1. Vim utilizes different modes to edit text.	This allows for complex chains of actions and utilization of solely the keyboard.
2. Pressing 'i' enters insert mode.	This is the primary manner of adding new text to a document.
3. Pressing 'Esc' or 'Ctrl-c' returns the user to normal mode.	This is required to switch back to navigation and command modes throughout Vim.
4. The keys 'h', 'j', 'k', and 'l' control directional cursor navigation.	This keeps navigation bound entirely to the keyboard home row, eliminating mouse reliance.
5. Pressing 'w' and 'b' advances the cursor forward and backward across words.	This enables rapid traversal through tokens and code identifiers without repeated keystrokes.
6. Pressing 'x' deletes the character currently under the cursor.	This allows quick single-character corrections without switching out of normal mode.
7. Pressing 'dd' deletes a line, while 'yy' copies it to the clipboard.	This executes line-level manipulation commands in a single streamlined, reproducible action.
8. Pressing 'p' pastes the most recently yanked or deleted text after the cursor.	This completes basic cut-and-paste operations entirely within normal mode.
9. Pressing 'u' undoes changes, while 'Ctrl-r' redoes them.	This provides essential error recovery controls directly through the keyboard interface.
10. Pressing ':' opens the command-line interface for ex-commands like ':w' and ':q'.	This executes core file operations such as saving and closing buffers without menu navigation.